

Final Project Report

Internship Management System

CS-229T Database Systems

Instructor: Dr. Anum Akber

Team Members

Maleeha Khan (2024F-BCS-013)

Fizzah Ahmed (2024F-BCS-009)

Tahira Sohail (2024F-BCS-007)

Table of Contents

Right-click and select "Update Field" to refresh page numbers after opening.

Table of Contents.....	2
1. Introduction	4
1.1 Project Background	4
1.2 Purpose of This Report.....	4
1.3 Objectives of the Internship Management System	4
2. Project Overview	4
2.1 System Description.....	4
2.2 Key Stakeholders	5
2.3 Major System Functionalities	5
2.4 Technology Stack.....	5
3. Phase 1 — Design and Planning.....	6
3.1 Requirement Analysis.....	6
3.2 Entity Identification and ER Diagram	6
3.3 Database Schema Design	6
3.4 API Documentation and Frontend Prototypes.....	6
3.5 Phase 1 Challenges	6
3.6 Phase 1 Team Contributions	7
4. Phase 2 — Backend Implementation	8
4.1 Database Implementation.....	8
4.2 Backend API Development.....	8
4.3 Authentication Module	8
4.4 Core, Assessment, and Workflow Modules.....	8
4.5 API Endpoint Summary	8
4.6 Role-Based Access Control.....	9
4.7 Testing and Validation	9
4.8 Phase 2 Challenges	9
4.9 Phase 2 Team Contributions	10
5. Phase 3 — Frontend Integration, Testing & Deployment.....	11
5.1 Authentication Integration	11
5.2 Core, Assessment, and Workflow Module Integration.....	11
5.3 API Testing and Validation	11
5.4 Role-Based Access Control Verification.....	11

5.5 Error Handling Verification	12
5.6 System Deployment Preparation	12
5.7 Phase 3 Challenges	12
5.8 Phase 3 Team Contributions	13
6. Overall Team Contributions Summary.....	13
6.1 Maleeha Khan (2024F-BCS-013)	13
6.2 Fizzah Ahmed (2024F-BCS-009).....	13
6.3 Tahira Sohail (2024F-BCS-007)	13
7. Conclusion	13

1. Introduction

1.1 Project Background

The Internship Management System is a comprehensive web-based platform designed to streamline and automate the process of managing student internships within academic institutions. Universities face significant challenges coordinating internship placements, tracking student progress, evaluating performance, and managing the administrative overhead associated with these programs. Traditional manual processes involving paper-based applications, email communications, and spreadsheet tracking are inefficient, error-prone, and lack transparency for all stakeholders involved.

This project was developed as part of the CS-229T Database Systems course, under the supervision of Dr. Anum Akber, with the objective of demonstrating practical application of database design principles, system architecture, and software engineering best practices across the complete software development lifecycle.

1.2 Purpose of This Report

This final report consolidates the work completed across all three project phases into a single, unified document. Phase 1 covered system design and planning, Phase 2 covered backend implementation, and Phase 3 covered frontend integration, testing, and deployment preparation. Together, these phases represent the complete journey of the Internship Management System from concept to a fully integrated, production-ready application.

1.3 Objectives of the Internship Management System

- To provide a unified platform for students to browse, apply for, and track internship opportunities offered by registered employers.
- To enable employers to post internship listings, review applications, evaluate student performance, and manage the hiring workflow efficiently.
- To empower academic coordinators with tools for overseeing the entire internship lifecycle, including application approvals, progress monitoring, workflow management, and credit verification.
- To automate administrative processes such as application status tracking, evaluation scheduling, viva assessment coordination, and credit hour verification.
- To maintain comprehensive digital records of all internship activities, ensuring data integrity, accessibility, and auditability.

2. Project Overview

2.1 System Description

The Internship Management System is a multi-role web application built on a three-tier architecture: a React-based frontend presentation layer, a Flask (Python) REST API application layer, and a PostgreSQL relational database layer. The system implements role-based access control with three distinct user categories: Students, Employers, and Coordinators, each with

functionalities tailored to their responsibilities. JWT-based authentication ensures protected access to sensitive information, with all endpoints (except registration and login) requiring a Bearer token in the Authorization header.

2.2 Key Stakeholders

Students: Students can browse available internship listings filtered by their department, submit applications with cover letters and resumes, track application status, view their assigned internships, submit progress reports, participate in viva assessments, and monitor their credit verification status.

Employers: Employers can register their companies, post internship listings with detailed requirements, review incoming applications, accept or reject applicants, evaluate intern performance, schedule and conduct viva assessments, and manage their hired interns throughout the internship period.

Coordinators: Academic coordinators have oversight across all activities within their department. They can approve or reject applications, monitor internship workflows, review and approve student progress reports, verify completed credit hours, and generate comprehensive reports on internship activities.

2.3 Major System Functionalities

The system encompasses ten major functional modules: Authentication and User Management, Student Profile Management, Employer Profile Management, Internship Listing Management, Application Processing, Internship Assignment and Tracking, Performance Evaluation, Viva Assessment Management, Workflow Progress Tracking, and Credit Hour Verification. Each module is designed with comprehensive CRUD operations, role-based access patterns, and proper data validation mechanisms.

2.4 Technology Stack

- Frontend: React with responsive UI components
- Backend: Flask (Python) with modular route organization
- Database: PostgreSQL with full constraint implementation
- Authentication: JWT tokens with Bearer scheme
- File Storage: Supabase Storage for document uploads
- API Design: RESTful with JSON data format, base URL `http://127.0.0.1:5000/api`

3. Phase 1 — Design and Planning

Phase 1 focused on the foundational design and planning activities required before implementation: requirement analysis, identification of all system entities and relationships, design of the Entity Relationship Diagram (ERD), creation of the detailed database schema, preparation of API documentation, and development of frontend interface prototypes.

3.1 Requirement Analysis

A thorough requirement analysis was conducted to identify all functional and non-functional requirements of the system, including stakeholder analysis, use case modeling, and functional decomposition. Key requirements identified include user authentication with role-based access, CRUD operations for all entities, application workflow management, automated status transitions, file upload capabilities, and comprehensive reporting features.

3.2 Entity Identification and ER Diagram

Ten core entities were identified: Users (base authentication entity), Students, Employers, Coordinators, Internship Listings, Applications, Internships, Evaluations, Viva Assessments, Workflow, and Credit Verifications. The Entity Relationship Diagram specifies cardinality constraints (one-to-one, one-to-many, many-to-many) and identifies primary and foreign keys. The Users entity serves as the central authentication hub with a one-to-one relationship to Students, Employers, and Coordinators. Employers post Internship Listings, which Students apply to through Applications (a many-to-many relationship resolved via a junction table). Accepted applications automatically generate an Internship record, and a Workflow entity tracks each student's progress through a seven-stage internship lifecycle.

3.3 Database Schema Design

The database schema translates the ER diagram into eleven relational tables, following Third Normal Form (3NF) to eliminate redundancy while preserving query performance. Primary keys use auto-incrementing SERIAL types; foreign key constraints maintain referential integrity with appropriate CASCADE policies; NOT NULL and UNIQUE constraints protect data integrity; and CHECK constraints validate enumerated fields such as role, status, and assessment_type.

3.4 API Documentation and Frontend Prototypes

Complete API documentation was prepared for all ten functional modules, specifying HTTP methods, URL patterns, authentication requirements, request/response formats, and status codes. Frontend prototypes were designed for the dashboard, application forms, and listing browser interfaces, establishing the layout, navigation, and interaction patterns later carried into implementation.

3.5 Phase 1 Challenges

Reconciling varying stakeholder expectations into a coherent requirement set required detailed use case mapping. Modeling the many-to-many relationship between students and listings was resolved with a junction table carrying additional status and approval attributes. Maintaining consistent API documentation across contributors was addressed through shared templates and style guidelines.

3.6 Phase 1 Team Contributions

Team Member	Primary Responsibilities	Deliverables
Maleeha Khan (2024F-BCS-013)	Requirement Analysis, System Architecture, API Docs (Auth, Students, Employers)	Requirements Document, Architecture Design, API Documentation
Fizzah Ahmed (2024F-BCS-009)	Database Design, ER Diagram, Schema Design, API Docs (Listings, Applications, Internships)	ER Diagram, Database Schema, API Documentation
Tahira Sohail (2024F-BCS-007)	Frontend Prototyping, UI/UX Design, API Docs (Evaluations, Viva, Workflow, Credits)	UI Mockups, Frontend Prototype, API Documentation

4. Phase 2 — Backend Implementation

Phase 2 transformed the Phase 1 architectural blueprint into a functional system: implementing the complete database schema in PostgreSQL, building the Flask REST API backend, and establishing all endpoints with authentication, validation, and role-based access control.

4.1 Database Implementation

All eleven tables were physically created in PostgreSQL with proper data types, primary keys, foreign keys, unique constraints, check constraints, and default values as specified in the Phase 1 design. CHECK constraints validate enumerated values for role, department, application status, internship status, and assessment_type, while UNIQUE constraints prevent duplicate email registrations and enforce one-profile-per-user rules.

4.2 Backend API Development

The Flask-based REST API was implemented with over 50 endpoints across ten functional modules, each following consistent patterns for CRUD operations, authentication middleware, request validation, and error handling. The API returns standard HTTP status codes — 200, 201, 400, 401, 404, and 500 — with all error responses following a consistent JSON structure.

4.3 Authentication Module

The Authentication module implements POST /auth/register and POST /auth/login. Registration accepts name, email, password, and role with conditionally validated role-specific fields. Login returns a JWT token along with user metadata (role, name, id, profile_id, department or company name). All protected endpoints validate the Bearer token and enforce role-based access at both the route and data query levels.

4.4 Core, Assessment, and Workflow Modules

The Students, Employers, Listings, Applications, Internships, and Evaluations modules were implemented with full CRUD operations and role-scoped data access — for example, coordinators see only students from their department, and accepted applications automatically generate Internship records. The Viva Assessments module allows employers to schedule and grade assessments, and students to submit answers. The Workflow module implements the seven-stage lifecycle with sequential, validated stage advancement and a progress-report submission/approval cycle, while the Credit Verifications module manages the final stage of the internship lifecycle.

4.5 API Endpoint Summary

Module	Endpoints	Key Operations
Authentication	2	Register, Login (JWT)
Students	5	List (dept-filtered), Get, Create, Update, Delete
Employers	4	List with user info, Create, Update, Delete
Listings	5	Browse (dept-filtered), My listings, Create, Update, Delete

Applications	6	List, My apps, Employer view, Pending, Submit, Status, Approve
Internships	5	List (role-scoped), Create, Update, Delete, Assign
Evaluations	3	List (role-scoped), My evaluations, Submit
Viva	6	List, My viva, Schedule, Grade, Submissions, Submit
Workflow	7	List, My workflow, Advance, Reject, Report, Approve
Credit	4	List, Pending, Initiate, Verify

4.6 Role-Based Access Control

The following matrix, established in Phase 2 and carried through Phase 3 verification, summarizes access permissions across all endpoint groups:

Endpoint Group	Student	Employer	Coordinator
Auth	Yes	Yes	Yes
Listings (browse)	Yes (dept filtered)	Yes	Yes
Listings (create)	No	Yes	No
Applications (submit)	Yes	No	No
Applications (accept/reject)	No	Yes	No
Applications (dept-approve)	No	No	Yes
Evaluations (submit)	No	Yes	No
Viva (schedule/grade)	No	Yes	No
Viva (submit answers)	Yes	No	No
Workflow (advance/reject)	No	No	Yes
Progress reports (submit)	Yes	No	No
Progress reports (approve)	No	No	Yes
Credit verification	No	No	Yes

4.7 Testing and Validation

All endpoints were tested for correct request handling, response formatting, authentication enforcement, and error conditions, including required-field checks, email format validation, duplicate-record prevention, and foreign key constraint enforcement.

4.8 Phase 2 Challenges

Implementing automatic internship creation on application acceptance required careful transaction handling within a single operation. Sequential workflow progression required server-side validation to prevent invalid state transitions. Consistent role-based data scoping across all

endpoints was achieved through middleware that extracts role and department from the JWT token and applies it at the query level.

4.9 Phase 2 Team Contributions

Team Member	Primary Responsibilities	Deliverables
Maleeha Khan (2024F-BCS-013)	Backend Architecture, Authentication API, Students API, Employers API	Flask App Structure, Auth Module, Core API Endpoints
Fizzah Ahmed (2024F-BCS-009)	Database Implementation, Listings API, Applications API, Internships API	PostgreSQL Schema, CRUD Endpoints, Business Logic
Tahira Sohail (2024F-BCS-007)	Evaluations API, Viva API, Workflow API, Credit Verification API	Assessment Endpoints, Workflow Logic, API Documentation

5. Phase 3 — Frontend Integration, Testing & Deployment

Phase 3 connected the React frontend to the Flask REST API, conducted end-to-end testing of the complete internship lifecycle, verified role-based access control and error handling across all interfaces, and prepared the system for production deployment.

5.1 Authentication Integration

The registration form connects to POST /auth/register with role-specific fields and inline validation, while the login page integrates POST /auth/login, storing the returned JWT in local storage and attaching it to all subsequent requests via the Authorization header. The login response determines the user's role and routes them to the appropriate dashboard.

5.2 Core, Assessment, and Workflow Module Integration

All ten functional modules — Students, Employers, Listings, Applications, Internships, Evaluations, Viva Assessments, Workflow, and Credit Verifications — were integrated into the frontend with role-specific data access. Reusable data table components accept column configurations and data-fetching functions as props, allowing the same component to serve different roles with different data. The seven-stage workflow lifecycle is rendered as a visual tracker, and a dedicated file upload service manages the complete flow from selection to Supabase Storage URL generation for cover letters, resumes, progress reports, and viva materials.

5.3 API Testing and Validation

All 50+ API endpoints across ten functional modules were re-verified for correct frontend integration, covering request handling, response formatting, authentication enforcement, and error conditions.

Module	Endpoints	Key Operations
Authentication	2	Register, Login (JWT)
Students	5	List (dept-filtered), Get, Create, Update, Delete
Employers	4	List with user info, Create, Update, Delete
Listings	5	Browse (dept-filtered), My listings, Create, Update, Delete
Applications	6	List, My apps, Employer view, Pending, Submit, Status, Approve
Internships	5	List (role-scoped), Create, Update, Delete, Assign
Evaluations	3	List (role-scoped), My evaluations, Submit
Viva	6	List, My viva, Schedule, Grade, Submissions, Submit
Workflow	7	List, My workflow, Advance, Reject, Report, Approve
Credit	4	List, Pending, Initiate, Verify

5.4 Role-Based Access Control Verification

Role-based access control was thoroughly verified at both the API and frontend levels. Frontend routes implement role-based guards: students are redirected from employer/coordinator pages, employers cannot access student submission or coordinator approval pages, and coordinators have exclusive access to workflow management and credit verification interfaces.

Endpoint Group	Student	Employer	Coordinator
Auth	Yes	Yes	Yes
Listings (browse)	Yes (dept filtered)	Yes	Yes
Listings (create)	No	Yes	No
Applications (submit)	Yes	No	No
Applications (accept/reject)	No	Yes	No
Applications (dept-approve)	No	No	Yes
Evaluations (submit)	No	Yes	No
Viva (schedule/grade)	No	Yes	No
Viva (submit answers)	Yes	No	No
Workflow (advance/reject)	No	No	Yes
Progress reports (submit)	Yes	No	No
Progress reports (approve)	No	No	Yes
Credit verification	No	No	Yes

5.5 Error Handling Verification

The frontend correctly interprets all HTTP status codes: 200 triggers a data refresh, 201 navigates to the new resource, 400 displays validation messages in form error banners, 401 triggers automatic logout and redirect to login, 404 displays a not-found message, and 500 displays a generic server error message. All error responses follow the consistent JSON format `{ "error": "Human-readable message" }`, parsed by the frontend error handling service for toast notifications and form banners.

5.6 System Deployment Preparation

The backend was configured for production with externalized base URL settings, CORS policies for the production frontend origin, environment-variable JWT secrets, and production PostgreSQL connection strings. The React frontend was built with optimized bundles and production API configuration. The deployed system requires a web server for the React frontend, a Python runtime for Flask, a PostgreSQL database server, and Supabase Storage for file uploads.

5.7 Phase 3 Challenges

Adapting UI components to both user role and API response structure was addressed with reusable, prop-configurable data tables. Maintaining authentication state across sessions required an authentication context provider managing token storage and login/logout functions.

Coordinating file selection with Supabase Storage upload and subsequent API submission required a dedicated upload service handling the complete flow.

5.8 Phase 3 Team Contributions

Team Member	Primary Responsibilities	Deliverables
Maleeha Khan (2024F-BCS-013)	Frontend Architecture, Auth Integration, Core UI	React App Structure, Auth Flow, Dashboard
Fizzah Ahmed (2024F-BCS-009)	Listings Integration, Applications UI, Internships	Listing Browser, Application Forms, Data Tables
Tahira Sohail (2024F-BCS-007)	Evaluations, Viva, Workflow, Credit UI	Assessment Interfaces, Workflow Viz, Report

6 Overall Team Contributions Summary

Across all three phases, each team member maintained consistent ownership of their assigned domain — architecture and authentication, data and core business logic, or assessment and workflow features — while collaborating closely on integration, documentation, and testing.

6.1 Maleeha Khan (2024F-BCS-013)

Maleeha led requirement analysis and system architecture in Phase 1, implemented the Flask application structure and JWT authentication in Phase 2, and integrated the authentication flow and dashboard into the React frontend in Phase 3 — owning the Authentication, Students, and Employers modules end-to-end.

6.2 Fizzah Ahmed (2024F-BCS-009)

Fizzah designed the ER diagram and database schema in Phase 1, implemented the PostgreSQL database and the Listings, Applications, and Internships APIs in Phase 2, and integrated those same modules into the frontend in Phase 3, including the listing browser and application workflow UI.

6.3 Tahira Sohail (2024F-BCS-007)

Tahira designed the frontend prototypes and API documentation structure in Phase 1, implemented the Evaluations, Viva, Workflow, and Credit Verification APIs in Phase 2, and integrated those modules into the frontend in Phase 3, including the workflow visualization and credit verification dashboard. She also compiled the progress reports for all three phases.

7. Conclusion

The Internship Management System project has been completed successfully across all three phases. Phase 1 established a solid architectural foundation with comprehensive requirement analysis, an Entity Relationship Diagram, a normalized database schema, and complete API documentation. Phase 2 transformed that foundation into a fully functional backend, with a PostgreSQL database, a Flask REST API of over 50 endpoints, JWT-based authentication, and role-based access control. Phase 3 completed the system by integrating the React frontend with

the backend, verifying role-based access and error handling end-to-end, and preparing the application for production deployment.

The system now provides a complete, integrated platform for managing student internships. Students can browse and apply for positions, track applications, submit progress reports, and participate in viva assessments. Employers can post listings, review applications, evaluate interns, and schedule assessments. Coordinators have comprehensive oversight, with department filtering, workflow management, and credit verification capabilities.

Throughout the project, the team demonstrated effective and consistent collaboration, with each member owning a coherent set of modules across design, implementation, and integration. The Internship Management System is now ready for production deployment, delivering significant value to academic institutions by streamlining internship processes, improving transparency, and reducing administrative burden for all stakeholders involved.